

5-Bar SCARA v2 Onboarding Package

M3

Encoder + Sensor Fusion Edition | Engineering Roadmap

FIRMWARE LEAD — VISION NODE

1 | ROLE & OBJECTIVE

As **Firmware Lead — Vision Node (M3)**, you are the eyes of the SCARA system. Your mission covers two phases: **Phase 3** — build a deterministic QVGA HSV color-detection pipeline running on the **ESP32-CAM (OV2640)** with sub-80ms latency and centroid accuracy under 3mm; and **Phase 3B** — fuse that raw centroid measurement with a frame-differencing optical flow estimate inside a **2D Kalman filter**, producing a smoothed, predictive (X, Y, Vx, Vy) state that survives brief occlusions.

Primary Mission: Deliver a FusionPacket containing fused position and velocity to M4's ESP-NOW bridge with confidence scoring, enabling M2 to pre-position the arm before the target is stationary.

2 | HARDWARE & COMPONENTS REQUIRED

Component	Specification / Notes	Qty
ESP32-CAM Module (AI-Thinker)	OV2640 sensor; PSRAM mandatory for QVGA RGB565 buffers	1
OV2640 Camera (attached)	QVGA (320×240) PIXFORMAT_RGB565; lock exposure/gain/WB after calibration	1
LED Ring Light	Uniform illumination across workspace; eliminates ambient variation	1
9-Point Calibration Grid	3×3 dot pattern at known world coordinates; print on flat surface	1
USB-to-Serial (FTDI)	For flashing ESP32-CAM (no onboard USB); 3.3V logic	1
Tripod / Camera Mount	Rigid overhead mount; no vibration; < 0.1mm positional drift	1
30nF / 100nF Capacitors	Power supply decoupling at ESP32-CAM VCC pin	2+
Test Objects (colored)	Distinct HSV colors for calibration; flat, matte surface preferred	3+

3 | SOFTWARE & PLATFORMS REQUIRED

Tool / Library	Purpose	Source
Arduino IDE / PlatformIO	Primary firmware development for ESP32-CAM	arduino.cc
esp32-camera library (Espressif)	Camera init, PSRAM allocation, frame grab	GitHub: espressif/esp32-camera
ESP-NOW (built-in)	Transmit FusionPacket to M4's kinematic bridge	ESP32 core
PSRAM APIs (ps_malloc)	Mandatory for 153,600-byte RGB565 frame + 76,800-byte mask	Built-in to ESP32 core
Python + numpy + matplotlib	Plot raw centroid vs. Kalman output during tuning; compute RMS error	pip install
Serial Plotter (Arduino IDE)	Real-time visualization of Kalman state during calibration	Built-in
Git	Version control; M4 depends on FusionPacket struct header	git-scm.com

4 | AI INTEGRATION STRATEGY

Your work involves signal processing math (Kalman matrices, optical flow, HSV thresholding) and tight PSRAM memory management on a resource-constrained MCU. Use AI for numerical validation, parameter tuning, and pipeline bottleneck diagnosis:

Prompt 1 — Tune Kalman filter parameters:

"I have a 2D Kalman filter tracking an object on a conveyor (constant-velocity model). State = [x, y, vx, vy]. Measurement = [x, y] from HSV centroid (noise ~3mm stddev). Current params: Q_POS=0.5, Q_VEL=2.0, R_MEAS=4.0. The filter lags 3 frames behind fast-moving objects. Explain how changing each parameter affects lag vs. noise. Suggest new values and explain the tradeoff. Show the predict-update equations."

Prompt 2 — HSV thresholding for your target color:

"I need to detect a red object under an LED ring on an ESP32-CAM OV2640. The camera outputs RGB565. Describe the optimal HSV range for red (wraps around 0/180 in OpenCV). Provide integer-math-only C++ code to convert a single RGB565 pixel to HSV without using floats in the inner loop. How do I handle the red hue wraparound?"

Prompt 3 — Memory budget analysis:

"ESP32-CAM with 4MB PSRAM. I need: QVGA RGB565 frame=153,600 bytes, binary mask=76,800 bytes, two 32x32 ROI buffers=2048 bytes each, KalmanState struct, camera DMA buffers. List all allocations, check if they fit in PSRAM, and flag any that must NOT use ps_malloc. What happens if fb_location is CAMERA_FB_IN_DRAM instead of PSRAM?"

Prompt 4 — Occlusion recovery logic review:

"Review this occlusion handling code: [paste your vision_loop_iteration()]. Identify: (1) what happens if occlusion_frames exceeds MAX_OCCLUSION_FRAMES, (2) whether confidence score correctly gates transmission, (3) any memory leak if esp_camera_fb_return() is missed in an early return path."

5 | BASIC ARCHITECTURE & CONNECTIONS

Input From	Your Processing	Output To
OV2640 camera (PSRAM frame)	RGB565→HSV filter → binary mask → centroid (u,v)	Kalman filter update step
Previous frame ROI buffer	32×32 block match → optical flow (dx,dy) estimate	Fuse into Kalman vx,vy
Kalman state (predict step)	Constant-velocity propagate before each frame	Predicted X,Y during occlusion
Calibration grid (9 points)	pixel_to_world scale_x, scale_y, offset_x, offset_y	M4: correct FusionPacket units
M5 reviewer (TEST 3B-A to E)	Tune Q_POS, Q_VEL, R_MEAS until all tests pass	Phase 3B gate sign-off
Your FusionPacket output	ESP-NOW transmit to M4	M2: pre-positioning + pick

Vision Loop: esp_camera_fb_get() [PSRAM] → HSV mask → blob gate → centroid (u,v) → pixel_to_world(wx,wy) → optical flow ROI → kalman_predict() → kalman_update() → confidence score → transmit_fused()

6 | TIMELINE & MILESTONES

[DAY 1]

- Set up ESP32-CAM flashing rig (FTDI adapter, IO0→GND for bootloader mode)
- Verify PSRAM is detected: log esp_get_free_heap_size() and esp_psram_get_size()
- Run camera demo: capture QVGA frame, print pixel count to Serial
- Sync with M4 on FusionPacket struct — agree on byte layout before Phase 3

[DAYS 2–5 (Phase 3 — HSV Pipeline)]

- Implement RGB565→HSV conversion in integer math (no floats in inner loop)
- Write HSV threshold filter: populate binary mask in PSRAM g_mask_buf
- Implement blob gate: reject blobs < 200px or > 30,000px
- Compute integer centroid: accumulator sum / pixel_count → (u,v)
- Implement pixel_to_world() transform with scale + offset
- Lock camera exposure, gain, white balance after initial calibration
- TEST 3-A: Pipeline latency < 80ms (millis() timing)
- TEST 3-B: 50 frames, centroid stddev < 2mm, mean error < 3mm
- TEST 3-C: Zero false positives in 100 frames with no object
- TEST 3-D: 9-point calibration grid RMS error < 3mm
- M4 reviews Phase 3 — obtain sign-off

[DAYS 6–9 (Phase 3B — Kalman + Optical Flow)]

- Allocate g_roi_prev and g_roi_curr in PSRAM (32×32 = 2048 bytes each)
- Implement estimate_flow(): extract V channel in ROI, SSD block match, return (dx,dy)
- Implement KalmanState struct: x,y,vx,vy,P[4][4],initialized,occlusion_frames
- Implement kalman_predict(): propagate state + covariance with Q_POS, Q_VEL
- Implement kalman_update(): innovation, Kalman gain, Joseph form covariance update
- Implement occlusion handling: dead-reckoning up to 3 frames, confidence decay
- Implement check_kalman_health(): reset on P[0][0] or P[1][1] > 100.0 mm²
- Tune Q_POS=0.5, Q_VEL=2.0, R_MEAS=4.0 — adjust by plotting Serial output

- TEST 3B-A: Stationary object, 50 frames, fused stddev < 1mm
- TEST 3B-B: 3-frame occlusion recovery within 5mm of truth
- TEST 3B-C: Moving object ~20mm/s, Vx/Vy within 30% of actual
- TEST 3B-D: Full fusion pipeline latency < 100ms total
- TEST 3B-E: No false transmit from centroid noise without object
- M5 reviews Phase 3B — obtain sign-off

7 | ESCALATION MATRIX

Issue / Blocker	First Contact	Second Contact
PSRAM allocation fails (ps_malloc returns NULL)	Verify board selection: AI-Thinker with PSRAM enabled	M6 — hardware audit
HSV threshold misses target color under LED ring	Retune with LED ring ON, ambient OFF; use AI Prompt 2	M5 — re-run TEST 3-B with new values
Kalman filter lags fast objects	Increase Q_POS and Q_VEL; use AI Prompt 1 for guidance	M5 — validate after retuning
Centroid RMS error > 3mm on calibration grid	Check camera tilt; recompute scale_x, scale_y	M4 — integration calibration step
Full pipeline > 80ms	Profile each stage; integer HSV must be in inner loop	M2 — check if comms are blocking vision core
FusionPacket struct mismatch with M4	M4 — sync header file; recompile both nodes	M6 for conflict resolution
Phase gate blocked by M5	M5 directly; share Serial plotter captures for async review	M6 for escalation

You are giving the robot the ability to see and predict. The Kalman filter you write will allow the arm to reach for objects before they stop moving. That's not just engineering — that's intelligent automation. Build it with precision and curiosity. ■■